



Le génie pour l'industrie

Département de génie logiciel et des TI

UQÀM | Département d'informatique

Assurance qualité

Professeur Alain April, ing. PHD

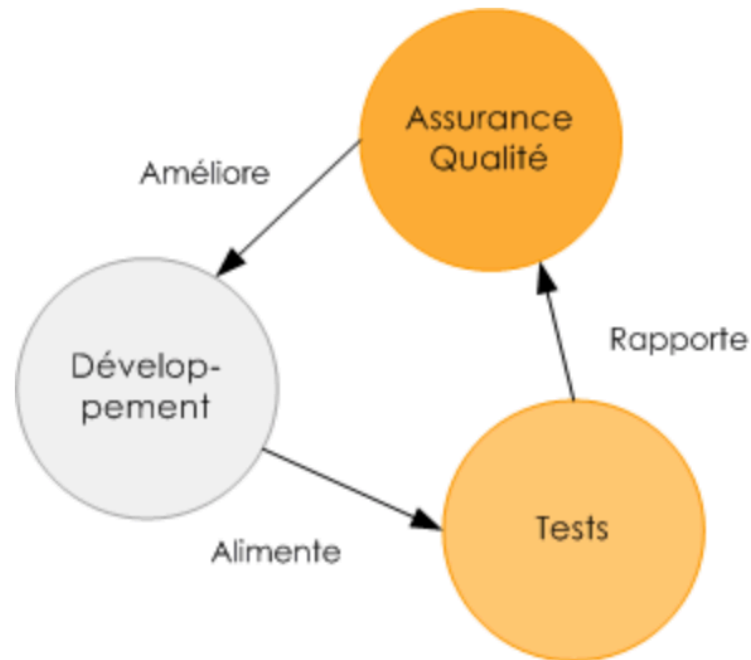
(révisé par la professeure Sylvie Trudel et Bruel Géranson, Ph.D., UQAM)



Le contenu de ce document est protégé par le droit d'auteur. Ce document peut être utilisé à des fins d'étude personnelle seulement. Aucune partie de cette œuvre ne peut être transmise ou reproduite d'une quelconque façon sans la permission du détenteur du droit d'auteur.

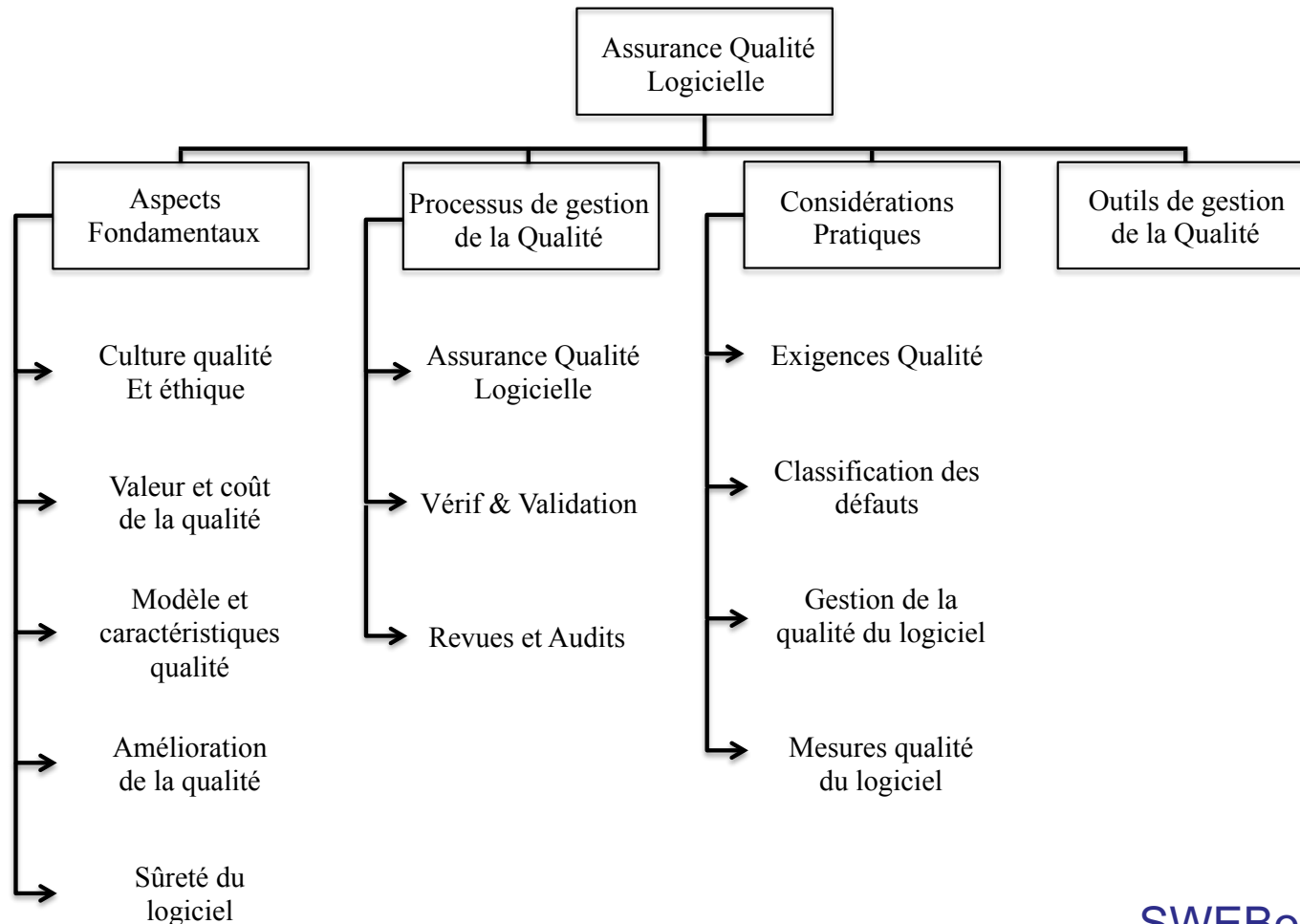


Section 2 – l'AQL



Dessin: © Les solutions applicatives Inc., 2010-2020

1.5 – Comment définir la qualité du logiciel?



SWEBOK



Définition: Logiciel



- Un ensemble de programmes, de procédures et de documentation associée et les données qui concernent le fonctionnement d'un système informatique (ISO 24765*):
 - Programmes : Qui se traduit en code source qui a fait l'objet de spécification, de conception, d'inspection et d'essais unitaires, système et d'acceptation avec la clientèle;
 - Données : Qui ont été inventoriées, modélisées, normalisées et créées pour effectuer des scénarios d'essais lors de sa création ou d'une modification;
 - Processus : Qui sont les processus d'affaires des utilisateurs et les processus qui ont été décrits (avant l'automatisation et après), étudiés et optimisés;
 - Règles : Qui sont des règles d'affaires qui ont dû être décrites, validées, implantées et testées;
 - Documentation : Toute sorte de documentation est utile pour les utilisateurs, développeurs et mainteneurs de logiciel. La documentation permet aux différents intervenants d'une équipe de mieux communiquer, réviser et tester le logiciel. La documentation est définie et réalisée à toutes les étapes clés du cycle de vie d'un logiciel;



Définition: Logiciel

- Le terme «logiciel» comprend le *firmware*, la documentation, les données et les instructions de contrôle d'exécution (*execution control statements (e.g., command files, job control language, etc.)*) (IEEE 730-2002)



IEEE P730™/D0.7 Draft Standard
for Software Quality Assurance
Processes

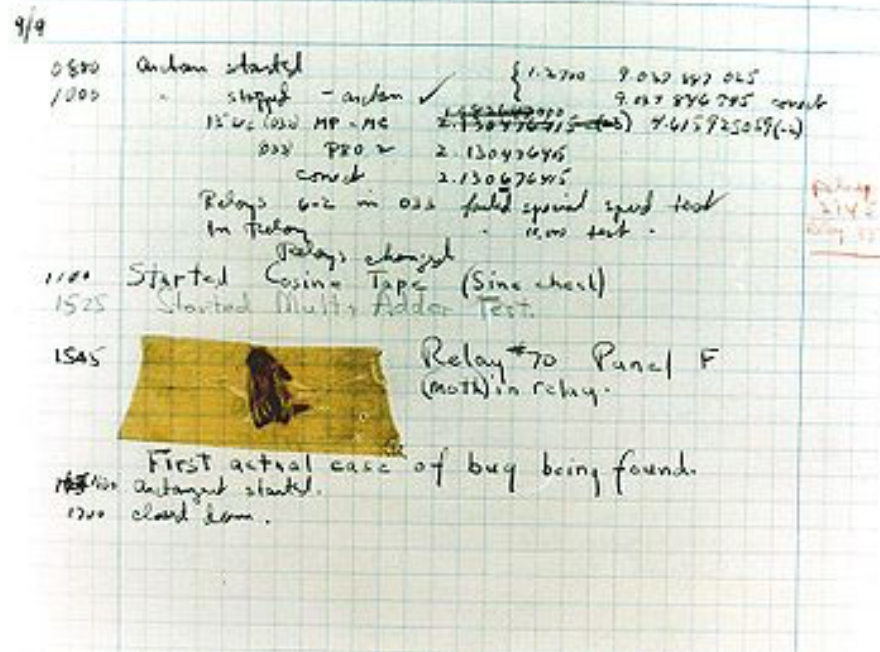
- Software includes the software portion of firmware. The software cannot be readily modified under program control* (ISO 12207)
- Micro logiciel (*Firmware*)**
 - Combinaison d'un dispositif matériel et d'instructions d'un ordinateur ou de données informatiques qui résident, en mode lecture seulement, sur un périphérique matériel (ISO 24765).

1.6 - Les erreurs, fautes et défaillances du logiciel

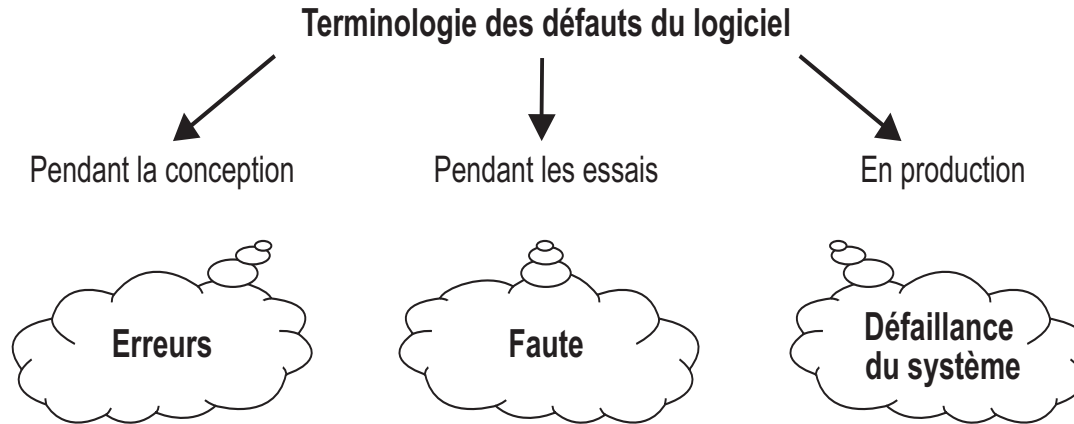
- Termes utilisés couramment pour décrire un problème dans notre domaine:
 - le système a planté en production;
 - le concepteur a fait une erreur;
 - à la suite d'une inspection (c.-à-d. une revue), on a trouvé un défaut dans le plan de test;
 - j'ai trouvé un « bogue » ;
 - le système est tombé en panne;
 - le client se plaint d'un problème avec un calcul dans le rapport de paiement;
 - on rapporte la défaillance du sous-système de surveillance.

Le mot « Bogue » à éviter

- Depuis l'époque de Thomas Edison, des ingénieurs ont utilisé le mot « bogue » pour référer à des failles dans les systèmes qu'ils ont développés. Ce mot couvre une multitude de problèmes possibles.
- Le premier cas documenté de « Bogue informatique » concernait un papillon de nuit (mite) coincé dans le relai de l'ordinateur Mark II de Harvard en 1947. L'opératrice de l'ordinateur, Grace Hopper, colla la mite dans le rapport de problème sous le titre "*First actual case of a bug being found*"



Les grandes catégories de causes d'erreurs du logiciel

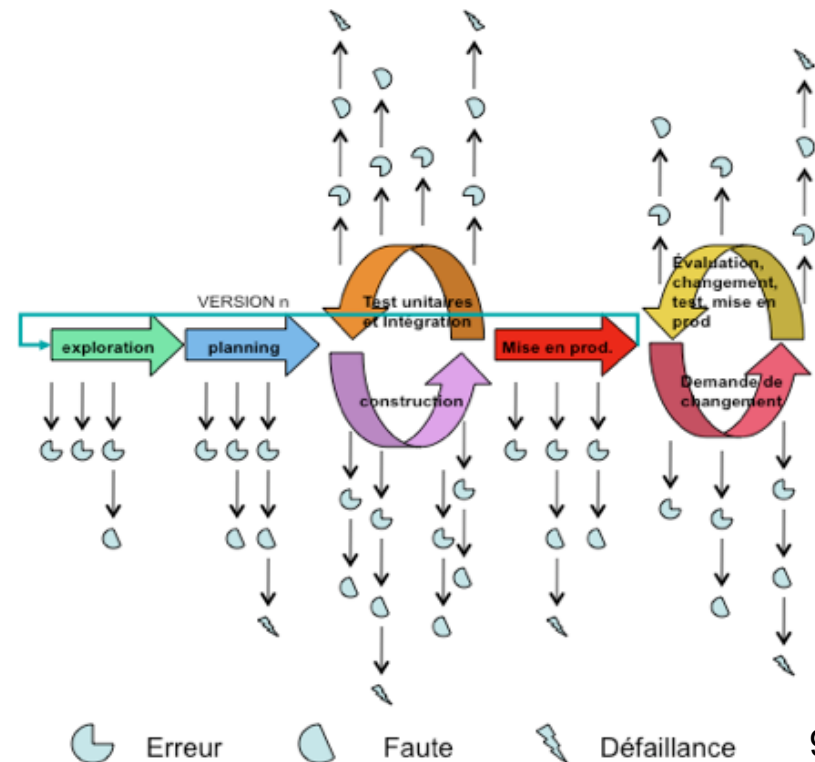


- **Erreur** (*error*): Une action humaine qui produit un résultat incorrect, cette erreur se cache dans les documents et le logiciel (ISO 24765).
- **Défaut/Faute** (*defect*): Le logiciel programmé avec cette erreur = une faute qui, si elle n'est pas corrigée, pourra causer une défaillance (*failure*) ou produire des résultats incorrects (ISO 24765).
- **Défaillance** (*failure*): La manifestation d'un défaut dans le logiciel (ISO 24765)
 - Cessation de l'aptitude d'un produit à accomplir une fonction requise ou de son incapacité à s'en acquitter à l'intérieur des limites spécifiées précédemment (ISO 25000).

À quel endroit introduit-on ces erreurs?

- Les erreurs peuvent être injectées dans toutes les étapes du cycle de vie du logiciel ;
- Ces défauts/fautes doivent être identifiés et corrigés avant de devenir des défaillances visibles par le client ;
- Les causes de ces défaillances, défauts/fautes et erreurs doivent être identifiées afin d'effectuer une correction.

C'est normal, en logiciel, de faire des erreurs. Il faut donc se l'avouer et prendre action !



Cas d'erreurs, fautes et de défaillances logiciels

Cas d'erreurs, fautes et de défaillances logiciels

Cas. La pharmacie **BeauPrix** ajoute une exigence logicielle à sa caisse enregistreuse afin d'empêcher une vente de plus de 75 dollars aux clients ayant une dette de plus de 200 dollars sur leur carte de crédit de la pharmacie. Le programmeur a mal compris la spécification et a créé un critère de vente défectueux de 500 dollars. Cette faute ne s'est jamais traduite en défaillance puisqu'aucun client n'a acheté pour plus de 500 dollars, étant donné que la carte de crédit offerte par cette pharmacie possède une limite de 400 dollars.

Quelles sont les source reconnues des erreurs dans un projet logiciel?

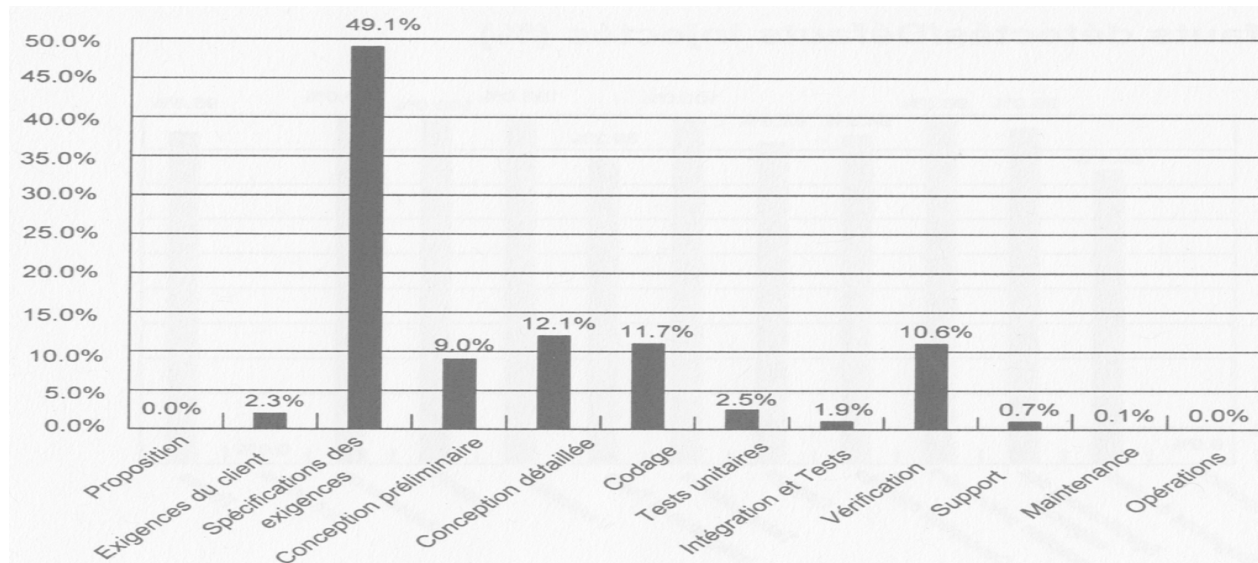
- Beizer présente une publication qui combine le résultat de plusieurs autres études concernant la provenance des erreurs:
 - 25 % Structurelles
 - 22 % Données
 - 16 % Fonctionnalités implémentées
 - 10 % Construction/Codage
 - 9 % Intégration
 - 8 % Exigences/Spécifications fonctionnelles
 - 3 % Définition/Exécution des tests
 - 2 % Architecture/Conception
 - 5 % Non spécifiées

BEIZER B., Software Testing Techniques, 2e édition, Van Nostrand Reinhold Co., New York, 1990.

Sources d'erreurs

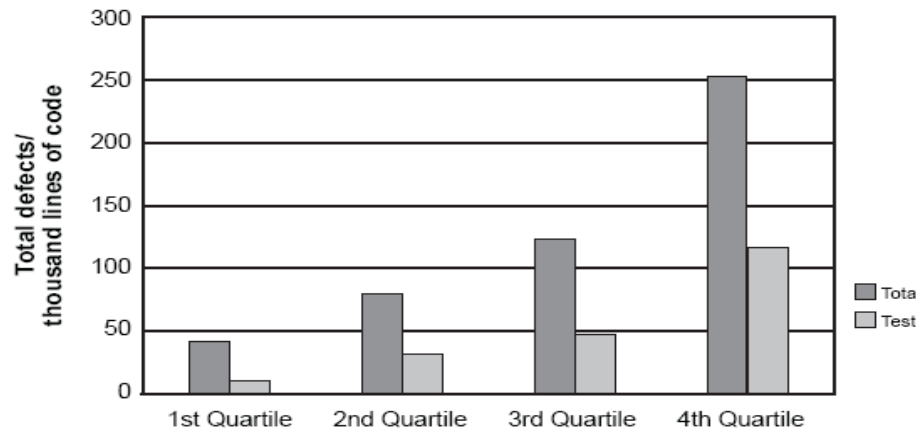
- Sources d'erreurs

Les sources d'erreurs



Étude: nombre de défauts injectés/détectés par les tests de 810 ingénieurs logiciels expérimentés

- Les développeurs injectent typiquement environ 100 défauts par 1000 lignes de code qu'ils écrivent
 - (c.-à.-d. 1 défaut/10 lignes de code)
- Il y a des variations considérables entre les développeurs
 - De 40 à 250 défauts injectés/1000 lignes de code



- Il observe des cas à 50 défauts/1000 lignes de code au début des tests
- Et d'autres cas de 10 à 20 défauts/1000 lignes au début des tests du système

Humphrey, W., The Software Quality Challenge, Crosstalk, juin 2008

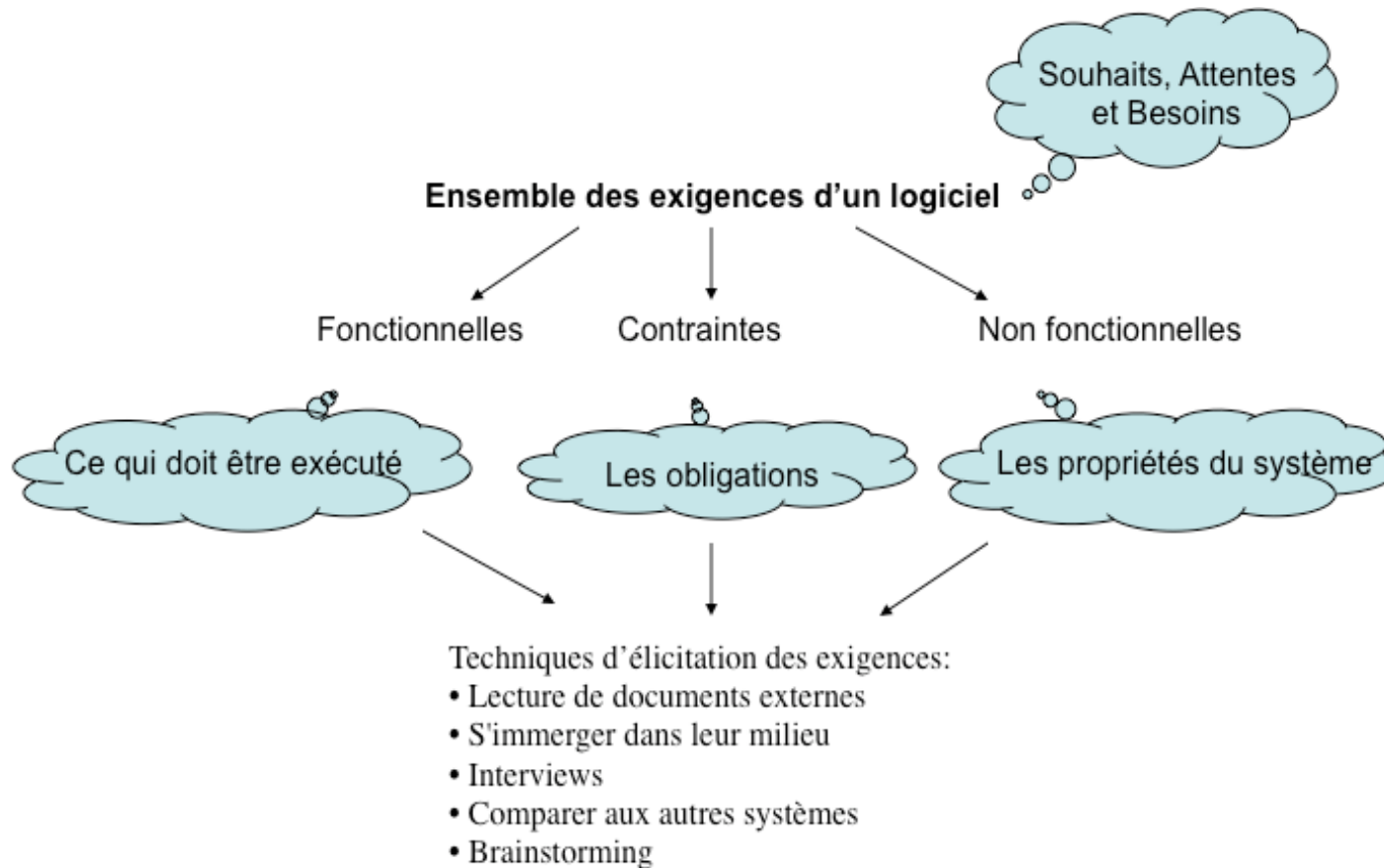
Noter la provenance des erreur (proposition de 8 catégories)

1. difficulté de définition des exigences;
2. difficulté de maintenir une communication efficace entre le client et le développeur;
3. déviations aux spécifications;
4. erreurs d'architecture et de conception;
5. erreurs de codage;
6. non-conformité avec les processus/procédures en place;
7. revues et tests inadéquats;
8. erreurs de documentation.

1 de 8) La difficulté de définition d'exigences

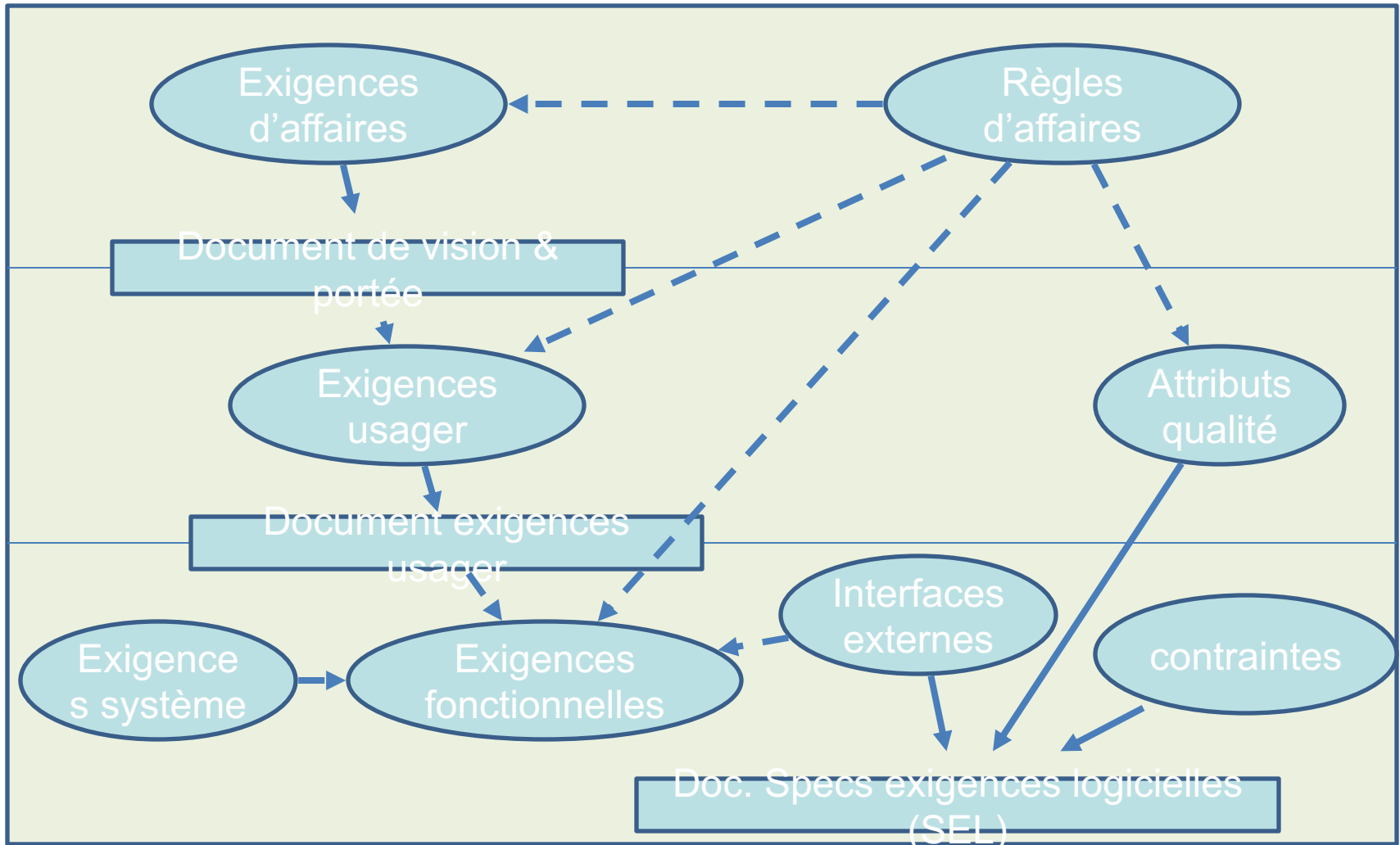
- Les enjeux du rôle de la personne qui joue le rôle d'analyste d'affaires (<https://montreal.iiba.org/fr>) - responsable de documenter les besoins et les exigences doivent maîtriser :
 - l'identification des intervenants concernés qui doivent participer;
 - la gestion des réunions;
 - la maîtrise des techniques d'interviews qui font le tri entre les souhaits, les attentes et les besoins réels;
 - la rédaction claire des exigences fonctionnelles, des obligations et des propriétés du futur système;
 - l'application systématique d'une technique d'élicitation des exigences;
 - la gestion des changements (aux exigences).

Concepts de définition* d'exigences

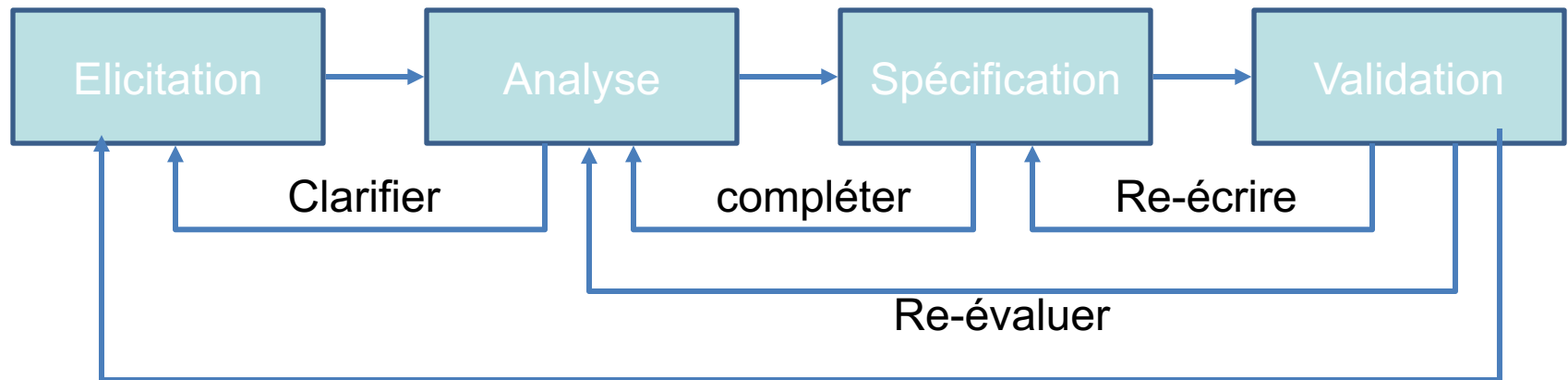


*** Élicitation ou élucidation = Obtention et explicitation**

Liens entre les différents types d'exigences



Définition des exigences



Pratiques pour l'élicitation

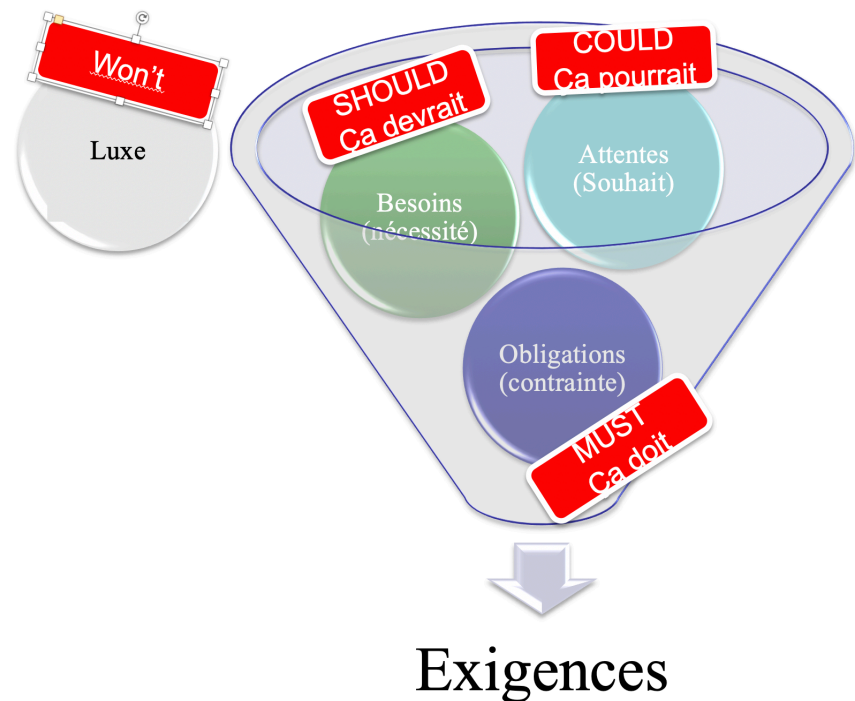
- **Définition de la vision et la portée du projet**
 - Ceci donne les exigences d'affaires, qui servent de point de référence pour évaluer les autres exigences
- **Identification des classes d'utilisateurs et leurs caractéristiques**
 - Pour éviter d'oublier des exigences. Caractériser par tâches et compétences, style de travail.
- **Sélection d'un 'product champion' dans chaque classe**
 - Représentant de la classe d'utilisateurs, mais aussi décideur pour cette classe
- **Travailler avec des représentants de classes d'utilisateurs pour éliciter les exigences**

La qualité d'une exigence logicielle

- Une exigence est dite de bonne qualité si elle rencontre les caractéristiques suivantes :
 - correcte;
 - complète;
 - claire pour chaque lecteur (p. ex. client, architecte, testeur);
 - concise (simple, précise);
 - consistante;
 - réalisable (réaliste, possible);
 - nécessaire (réponds à un besoin du client);
 - indépendante de la conception;
 - indépendante de la technique d'implantation;
 - vérifiable et testable;
 - unique.

Prioriser les exigences: MoSCoW

- **M** : une exigence essentielle, voire vitale, qui doit être réalisée « *Must have this* » ;
- **S** : une exigence importante « *Should have this if at all possible* » ;
- **C** : une exigence de confort (souhaitable) pourrait être réalisée dans la mesure où elle n'a pas d'impact sur les autres tâches « *Could have this if it does not affect anything else* » ;
- **W** : une exigence considérée un luxe qui ne sera pas réalisée maintenant, mais pourrait être réalisée plus tard « *Won't have this time but would like in the future* ».



2 de 8) La difficulté de maintenir une communication efficace

- Pour minimiser les erreurs :
 - prenez des notes à chaque réunion et diffusez le compte rendu à toute l'équipe du projet;
 - soyez cohérent dans votre utilisation de termes et faites-vous un glossaire des termes* que vous allez utiliser et assurez-vous que toutes les parties prenantes en ont une copie;
 - Informez les utilisateurs du cout des modifications des spécifications;
 - choisissez un cycle de vie de développement qui permet d'accepter des modifications en cours de route;
 - numérotez chaque exigence et instaurez la gestion du changement.

La difficulté de maintenir une communication efficace (suite)

- Les erreurs se glissent dans les produits intermédiaires du logiciel à cause, entre autres, de mésententes involontaires entre le personnel TI et les clients dès le démarrage des projets logiciels
 - une mauvaise compréhension ou interprétation des instructions des clients;
 - le client désire des résultats immédiats;
 - le client ne prend pas le temps de lire la documentation qui lui est envoyée;
 - mauvaise compréhension des changements demandés aux développeurs pendant la conception;
 - l'analyste arrête d'accepter des changements à la phase de conception.
 - Dans certains projets 25 % des spécifications auront changé avant la fin du projet.

3 de 8) Les déviations aux spécifications

- Quand le développeur interprète mal une spécification* et procède à la conception et au développement du logiciel avec sa compréhension du besoin.
 - Cette situation crée des défauts qui ne seront malheureusement capturés que tard dans le développement ou en opération (défaillance).
- D'autres types de déviations sont :
 - le développeur réutilise du code source existant sans effectuer les ajustements nécessaires pour rencontrer les nouvelles exigences;
 - à cause de pressions budgétaires ou de temps, le développeur décide de laisser tomber une partie des exigences afin de terminer son travail;
 - des initiatives et des 'améliorations' initiées par les développeurs sans consultation avec les clients.

4 de 8) Les erreurs d'architecture et de conception

- Erreurs typiques de conception:
 - conception qui n'utilise pas les bons algorithmes pour rencontrer les exigences;
 - élicitation erronée de séquence de processus;
 - mauvaise conception des critères des règles d'affaires;
 - omission d'états d'une transaction pour bien représenter le processus du client;
 - omission de traiter les erreurs et opérations illégales ce qui permet au logiciel de traiter des cas qui ne seraient pas existants dans les affaires du client :
 - on estime que jusqu'à 80 % du code d'un programme traite des cas exceptionnels ou d'erreurs.

5 de 8) Les erreurs de codage

- **Erreurs typiques de programmation:**
 - choix de langage inadéquat et de conventions de programmation;
 - ne pas adresser la gestion de la complexité dès le départ;
 - mauvaise compréhension/interprétation des documents de conception;
 - abstractions incohérentes;
 - erreurs dans des boucles et conditions;
 - erreur dans le traitement de données;
 - erreur dans la séquence du traitement;
 - manque ou mauvaise validation d'une donnée à l'entrée;
 - mauvaise conception des critères des règles d'affaires;
 - omission d'états d'une transaction pour bien représenter le processus du client;

6 de 8) La non-conformité aux processus/procédures en place*

- Certains organismes possèdent leur méthodologie interne propre et leurs normes internes pour la construction d'un logiciel. Cette méthodologie décrit:
 - Les processus, procédures, étapes, livrables, gabarits et normes qui encadrent l'acquisition, le développement, la maintenance et l'opération des logiciels
- Un employé qui ne fait que coder semble bien plus productif que celui qui développe l'ensemble des produits intermédiaires mandatés par la méthodologie interne:
 - Est-ce que l'employé est récompensé pour une plus grande production ?
 - Au détriment de la qualité de la documentation

La non-conformité aux processus/procédures en place (suite)

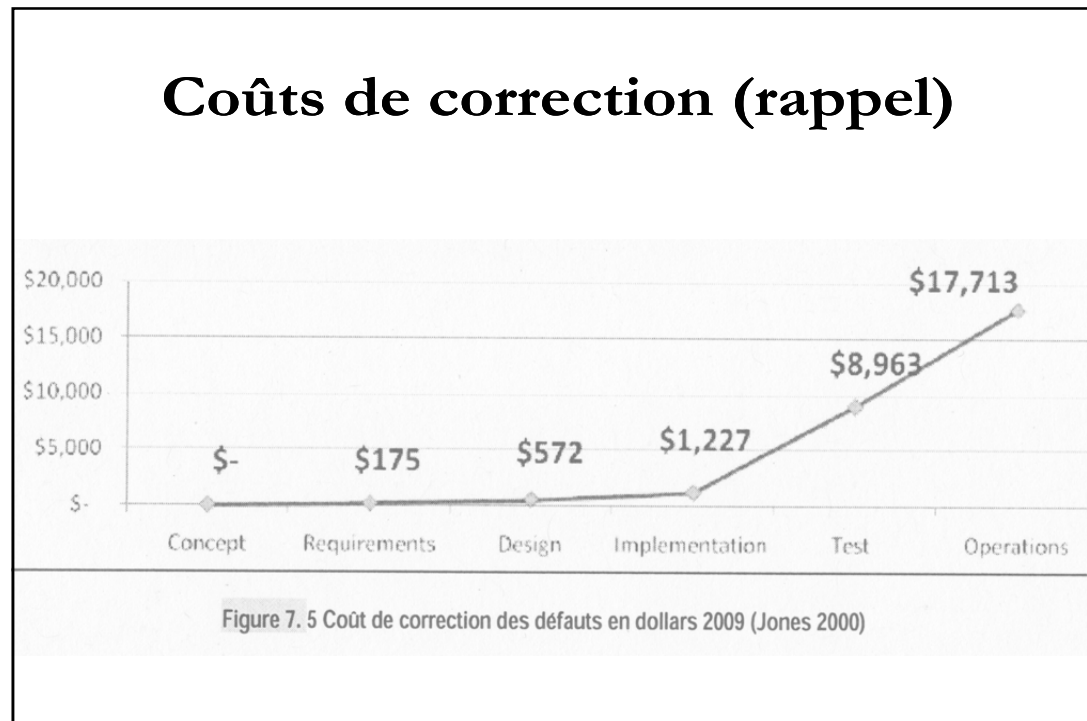
- Un logiciel, non documenté, soulèvera, tôt ou tard, les problématiques suivantes :
 - les membres de l'équipe TI ont besoin de coordonner leur travail avec les autres afin de servir l'ensemble de leurs utilisateurs; ces membres d'équipe vont soulever la difficulté de comprendre et de tester le logiciel de ce programmeur;
 - la personne qui va remplacer ou maintenir le logiciel développé n'a rien d'autre que le code source pour comprendre ce logiciel;
 - le personnel d'AQL trouvera un nombre important de non-conformités relatives à ce logiciel;
 - l'équipe de tests aura de la difficulté à développer des plans et des scénarios de tests, car les spécifications ne sont pas disponibles;
 - l'équipe de maintenance responsable de supporter l'application en production aura de la difficulté à comprendre le logiciel:

7 de 8) Les revues et les tests inadéquats

- Difficultés lors des revues et des tests:
 - les revues ne couvrent qu'une toute petite portion des livrables intermédiaires du logiciel;
 - les revues ne découvrent pas toutes les erreurs présentes dans la documentation et le code du logiciel;
 - la liste des recommandations des revues n'est pas implantée et suivie adéquatement;
 - des plans de tests incomplets ne permettent pas une couverture adéquate de l'ensemble du logiciel laissant des parties;
 - le projet n'a pas planifié beaucoup de temps pour effectuer les essais.
 - Dans certains cas, cette étape est raccourcie, car elle est coincée entre le codage et la livraison finale.
 - le processus des essais ne rapporte pas correctement les erreurs et défauts trouvés;
 - les défauts trouvés sont corrigés, mais ne font pas l'objet de tests adéquats par la suite:
 - Par exemple les tests de régression

Coûts de correction

Coûts de correction



8 de 8) Les erreurs de documentation

- Il est reconnu que la documentation obsolète et déficiente des logiciels qui sont en opération, dans nos organismes, est un problème courant;
- Peu d'équipes de développeurs aiment passer du temps à documenter.

1.7 – La qualité dans le domaine du logiciel



- Capacité d'un produit logiciel de satisfaire les besoins exprimés et implicites (c.-à-d. les attentes) quand il est utilisé dans des conditions spécifiées (ISO 24765);
- Il faut satisfaire les besoins et attentes des clients et utilisateurs
 - Les attentes ne sont pas nécessairement décrites;
 - Il y a souvent plusieurs ‘utilisateurs’:
 - L'utilisateur du logiciel;
 - Les personnes qui vont l'installer;
 - Les personnes qui vont le maintenir.

1.8 – L'assurance qualité logicielle



- 1) Un ensemble d'activités planifiées et systématiques de toutes les actions nécessaires pour fournir une assurance suffisante qu'un élément produit est conforme aux exigences techniques établies;
 - 2) Un ensemble d'activités destinées à évaluer le processus par lequel les produits sont développés ou fabriqués;
 - 3) Les activités planifiées et systématiques mises en œuvre dans le système qualité, et démontrées au besoin pour fournir une assurance suffisante qu'une entité satisfera aux exigences de qualité.
- **Notes:**
 - La définition du mot 'assurer'* signifie 'donner la certitude'.
 - *There are both internal and external purposes for quality assurance:*
 - *Within an organization, quality assurance provides confidence to management;*
 - *In contractual situations, quality assurance provides confidence to the customer or others.*

* Le terme 'assurance-qualité', comme le terme 'assurance-vie', peut porter à confusion, car la mise en place de pratiques de génie logiciel ne peut pas 'assurer' la qualité d'un projet.

L'assurance qualité

- L'assurance qualité est mise en place pour réduire les risques de développer un logiciel de faible qualité;
- Cette perspective de l'assurance qualité (AQ) implique, pour le développement du logiciel, les éléments suivants :
 - la nécessité de planifier les aspects qualité d'un produit ou d'un service;
 - le besoin de déployer des activités systématiques qui nous indiquent que, tout au long du cycle de vie du logiciel, certaines corrections sont requises;
 - que le système qualité est un dispositif complet qui doit permettre, dans le cadre de la gestion de la qualité, la mise en œuvre de la politique qualité et l'amélioration continue;
 - l'exécution de techniques d'assurance qualité qui ont pour objectif de démontrer le niveau de qualité atteint de manière à donner confiance aux utilisateurs et clients;
 - la démonstration de la satisfaction des exigences qualité qui ont été définies pour le projet, la modification ou le service TI.



Le contrôle de la qualité du logiciel

- Un ensemble d'activités visant à évaluer la qualité des produits développés ou fabriqués;
- Le processus de vérification de son propre travail ou de celui d'un collègue:
 - **NOTE** – Ce terme n'a pas de signification normalisée en génie logiciel en ce moment.

Assurance vs contrôle de qualité

Assurance vs contrôle de qualité

Contrôle de qualité

- Produit fini, contrôlé avant sa livraison

Assurance qualité

- Activités faites tout au long du processus de développement

Ces activités :

- Préviennent les erreurs
- Les détectent et les corrigent

NB: Le contrôle de qualité est une activité de l'assurance qualité

Coût de qualité

Coût de qualité

